

Moving a Proxmox Machine to Another Host and Wiping the Disk

Scenario: Sending out a desktop that currently runs Proxmox, without being able to maintain remote access or keep the SSD. The hardware will no longer be in possession, but the OS and its data are valuable enough to keep, and transfer to a new machine that currently has not been purchased.

Goal: Keep a clone of the entire Proxmox OS, and all of the data stored on it, with the ability to easily restore the OS on new hardware in the future. This is not a direct transfer to a new host, but just saving the data to a backup server.

Note: Tools such as ChatGPT and other sites were used in this process.

Table of Contents

- Moving a Proxmox Machine to Another Host and Wiping the Disk 1**
 - Scenario:..... 1*
 - Goal:..... 1*
 - Notes: 1*
- Planning and Backup 2**
 - Inventory 2*
 - Finding a backup location 2*
 - Proving the Network 3*
 - Preserving the Configs and Data..... 4*
 - Starting the Backups 6*
 - Moving the Proxmox OS Data 7*
- Sanitizing the Disk 8**

Planning and Backup

A plan was first established to accomplish this goal in two main jobs. First, was backing up and preserving the Proxmox OS and its VMs. Second, destroy the data on the drive using built-in secure erasing functionality.

Inventory

To determine the hardware on the machine, how Proxmox is storing data, and to collect information about the VMs, an inventory was created by running:

```
lsblk -o NAME,SIZE,MODEL,SERIAL,TYPE,FSTYPE,MOUNTPOINTS
pvesm status
qm list
pct list
```

This creates a visualization of the disk, VM status, and pool information. In this case, there was only one drive with three VMs and no pools, so this made the process rather simple.

Finding a backup location

On the same network, a Ubuntu server was running with a 2tb SSD, that was luckily acquired before prices skyrocketed. That drive was checked for available space and had a 1.8Tb ext4 partition that was 4% used. This information was obtained using:

```
lsblk -o NAME,SIZE,MODEL,SERIAL,TYPE,FSTYPE,MOUNTPOINTS
df -hT
```

```
~$ lsblk -o NAME,SIZE,MODEL,SERIAL,FSTYPE,MOUNTPOINTS
NAME        SIZE MODEL          SERIAL          FSTYPE MOUNTPOINTS
loop1       13.5M
loop2       74M
loop3       74M
loop4       66.8M
loop5       66.8M
loop6       60.1M
loop7       115.1M
loop8       122.2M
loop9       50.1M
loop10      50.1M
loop11      13.1M
sda         1.8T WD Blue SA510 2.
├─sda1      1G
└─sda2      1.8T
    ext4    /

~$ df -hT
Filesystem      Type      Size  Used Avail Use% Mounted on
tmpfs           tmpfs     788M  3.1M  785M   1% /
/dev/sda2       ext4      1.8T   64G  1.7T   4% /
tmpfs           tmpfs     3.9G   0  3.9G   0% /
tmpfs           tmpfs     5.0M   0   5.0M   0% /
/dev/sda1       vfat      1.1G   6.1M  1.1G   1% /
tmpfs           tmpfs     788M  4.0K  788M   1% /
```

Both hosts were placed on a Netgear 5-Port Unmanaged Gigabit Ethernet Switch, which was the newest hardware available. Finally, the 64Gb was opened to analyze its disk usage on that file system, going only one level deep. Errors were discarded and the results were sorted from smallest to largest.

```
sudo du -xhd1 / 2>/dev/null | sort -h
```

A directory was then created on the Ubuntu server to store the backup files, and ownership of that directory was transferred to the user. The new directory was also verified to have the necessary size of ~1.7Tb.

```
sudo mkdir -p /proxmox-backup
sudo chown $USER:$USER /proxmox-backup
df -h /proxmox-backup
```

Proving the Network

An optional step at this point was to test the viability of the network. There were no known issues, but the information could have proven to be useful. First, the IP addresses and masks of both hosts showed that they were on the same LAN. Then, a successful SSH connection was established from the Proxmox host to the Ubuntu server, indicating that tools such as SSH or SCP are working and available for use.

The next optional quick test was to determine the speed of the switch connecting the two hosts. This was accomplished using iperf3. On Linux, iperf3 is a command-line network performance testing tool that measures how much data can be transferred over a network. It also measures statistics such as throughput, latency, packet loss, bottlenecks, and more. On the Ubuntu server, the iperf3 package was installed and the listener was started.

```
sudo apt install iperf3
iperf3 -s
```

On the Proxmox host, iperf3 was installed and connected to the Ubuntu listener. This started the test to determine whether the data transfer would be able to maintain its integrity, and to estimate the potential speed of the network.

```
sudo apt install iperf3
iperf3 -c <Ubuntu IP>
```

The Netgear switch held up to its claims, and the transfer speeds were 900+Mbit/s.

```
[ ID] Interval          Transfer    Bitrate
[  5]  0.00-1.00    sec    108 MBytes  903 Mbits/sec
[  5]  1.00-2.00    sec    112 MBytes  941 Mbits/sec
[  5]  2.00-3.00    sec    112 MBytes  941 Mbits/sec
[  5]  3.00-4.00    sec    112 MBytes  941 Mbits/sec
[  5]  4.00-5.00    sec    112 MBytes  941 Mbits/sec
[  5]  5.00-6.00    sec    112 MBytes  941 Mbits/sec
[  5]  6.00-7.00    sec    112 MBytes  942 Mbits/sec
[  5]  7.00-8.00    sec    112 MBytes  941 Mbits/sec
[  5]  8.00-9.00    sec    112 MBytes  941 Mbits/sec
[  5]  8.00-9.00    sec    112 MBytes  941 Mbits/sec
- - - - -
[ ID] Interval          Transfer    Bitrate
[  5]  0.00-9.00    sec   1.06 GBytes 1.01 Gbits/sec
iperf3: the client has terminated
```

Preserving the Configs and Data

Using bash, an inventory was created of the existing Proxmox data, which was then subsequently sent to the Ubuntu server. The script created a neat file that displays the information in a decent manner. Also on the list was the Proxmox cluster configuration and VM Configuration files, which were also sent. Note that this was taken before any data was backed up, transferred, or erased.

```
{
    echo "==== HOST ====="
    hostnamectl

    echo
    echo "==== KERNEL ====="
    uname -a

    echo
    echo "==== PROXMOX VERSION ====="
    pveversion --verbose

    echo
    echo "==== STORAGE ====="
    lsblk -o NAME,SIZE,MODEL,SERIAL,TYPE,FSTYPE,MOUNTPOINTS

    echo
    echo "==== PVE STORAGE ====="
    pvesm status

    echo
    echo "==== VIRTUAL MACHINES ====="
    qm list

    echo
    echo "==== CONTAINERS ====="
    pct list

    echo
    echo "==== NETWORK ====="
    ip -br addr
    ip route
    /etc/default/grub
```

```

echo
echo "===== NETWORK CONFIG ====="
cat /etc/network/interfaces

echo
echo "===== FSTAB ====="
cat /etc/fstab

} > /tmp/proxmox-inventory.txt

scp /tmp/proxmox-inventory.txt user@<UBUNTU IP>:/proxmox-backup/

sudo tar -czf /tmp/proxmox-config.tar.gz \
/etc/pve \
/var/lib/pve-cluster \
/etc/network/interfaces \
/etc/hosts \
/etc/hostname \
/etc/fstab \
/etc/default/grub

scp /tmp/proxmox-config.tar.gz \
user@<UBUNTU IP>:/proxmox-backup/

```

```

sudo mkdir -p /tmp/pve-configs

sudo cp -a /etc/pve/qemu-server /tmp/pve-configs/
sudo cp -a /etc/pve/lxc /tmp/pve-configs/ 2>/dev/null || true

sudo tar -czf /tmp/pve-guest-configs.tar.gz -C /tmp pve-configs
scp /tmp/pve-guest-configs.tar.gz \
user@<UBUNTU IP>:/proxmox-backup/

```

Using sha256 hashes, the files that landed on the Ubuntu server were verified. The hashes on both sides came out the same, indicating that the files were not corrupted in transit.

```

sha256sum /tmp/proxmox-inventory.txt \
/tmp/proxmox-config.tar.gz \
/tmp/pve-guest-configs.tar.gz

```

```

ssh user@<UBUNTU IP> 'sha256sum /proxmox-backup/proxmox-inventory.txt /proxmox-
backup/proxmox-config.tar.gz /proxmox-backup/pve-guest-configs.tar.gz'

```

Starting the Backups

At this point it was time to start backing up the VMs. There were three VMs hosted on the Proxmox server, and one of them was running at the time. As an exercise, the machine was left on. It was okay to shut it down, but as a demonstration of possibility, it was left running. Proxmox has a live snapshot mechanism using `vzdump`. Since the full allocated disk was not used, it was compressed with `zstd`. This snapshot was taken using:

```
vzdump 100 --storage local --compress zstd --mode snapshot
```

Since there were existing snapshots for the other two machines, the fingerprints of each backup file could be established. First, the files were listed out, then the sha256 hash was created for each.

```
ls -lh /var/lib/vz/dump/  
sha256sum /var/lib/vz/dump/vzdump-qemu-*.vma.zst
```

The `vzdump` directory was then moved to the Ubuntu server via SCP, and the files were verified with sha256 hashes. The hashes were the same, so the transfer was successful.

```
scp /var/lib/vz/dump/* \  
    user@UBUNTU IP>:/proxmox-backup/vzdump/  
  
ssh user@UBUNTU IP> 'ls -lh /proxmox-backup/vzdump/'  
ssh user@UBUNTU IP> 'sha256sum /proxmox-backup/vzdump/*.vma.zst'
```

At this point, for the sake of double checking, another round of data collection occurred. This included VM state, `lsblk` data, and the `nvme` information. The VMs were all given the shutdown command and confirmed to be stopped.

```
qm status 100  
qm status 101  
qm status 102  
pvesm status  
lsblk -o NAME,SIZE,MODEL,SERIAL,TYPE,FSTYPE,MOUNTPOINTS  
sudo nvme list  
qm shutdown 100  
qm shutdown 101  
qm shutdown 102  
qm list
```

The backup hashes were also stored and sent to the Ubuntu server for future reference, not a required step in all scenarios, but may be useful in production or higher security environments. A quick check was also performed to ensure the file arrived correctly.

```
{  
    echo "==== PROXMOX NATIVE BACKUP SHA256 ==="  
    date  
    sha256sum /var/lib/vz/dump/*.vma.zst  
  
    echo  
    echo "==== CONFIG BACKUP SHA256 ==="  
    sha256sum /tmp/proxmox-config.tar.gz  
    sha256sum /tmp/pve-guest-configs.tar.gz  
    sha256sum /tmp/proxmox-inventory.txt  
} > /tmp/proxmox-backup-hashes.txt  
scp /tmp/proxmox-backup-hashes.txt user@UBUNTU IP>:/proxmox-backup/  
ssh user@UBUNTU IP> 'cat /proxmox-backup/proxmox-backup-hashes.txt'
```

From here, the Proxmox host was shut down, and an Ubuntu Live USB was created for the full drive transfer.

Moving the Proxmox OS Data

To move the contents of the operating system, it must first be shut down. A Live USB was used to manage the data on the drive from this point onward. A Linux Live USB is a simple USB drive that has a Linux ISO installed to it. The machine boots from this USB instead of the main hard drive. This allowed for the contents of the drive containing Proxmox to be transferred over to the Ubuntu Server.

From this live environment, the hardware checks were run again, and networking was also confirmed.

```
lsblk -o NAME,SIZE,MODEL,SERIAL,TYPE,FSTYPE,MOUNTPOINTS
ip -br addr
ping -c 4 <UBUNTU IP>
ssh user@<UBUNTU IP>
exit
```

The tools needed were zstd and nvme-cli, which were installed and verified. The nvme tool was then used to identify the specific hardware model and serial number of the drive. This information reveals the drive's built-in wiping abilities.

```
sudo apt install -y zstd nvme-cli
zstd --version
nvme version
sudo nvme list
sudo nvme id-ctrl /dev/nvme0n1 | grep -E 'mn|sn|fr'
```

An additional safety check was rerun to ensure that the Ubuntu server had enough room to store the data and that the destination directory was writable by the user. The Proxmox drive state was also brought up to check for errors or warnings, and to see what it has for sanitization capabilities. The last check was to see if there were any files on the Ubuntu server that would be overwritten, which returned nothing.

```
sudo nvme smart-log /dev/nvme0n1
sudo nvme id-ctrl /dev/nvme0n1 | grep -E 'oacs|sanicap|fna'
ssh user@<UBUNTU IP> 'ls -lh /proxmox-backup/<PROXMOX-NAME>-nvme0n1.img.zst
2>/dev/null || echo "No existing disk image -- good to go."'
```

After these checks, it was time for the big move, copying the entire drive over to the Ubuntu server. This was done using dd, compressing it with zstd, and sending it to the Ubuntu host over SSH.

```
sudo dd if=/dev/nvme0n1 bs=16M status=progress \
| zstd -T0 -3 \
| ssh user@<UBUNTU IP> \
'cat > /proxmox-backup/<PROXMOX-NAME>-nvme0n1.img.zst'
```

After the lengthy transfer was complete, the backup was verified with zstd, and a sha256 hash was generated for the entire file. At this point, the backups were created and immediately validated. The files were confirmed to be in a stored state where the data could be put onto a new host in the future.

```
ssh user@<UBUNTU IP> \
'ls -lh /proxmox-backup/<PROXMOX-NAME>-nvme0n1.img.zst && \
zstd -t /proxmox-backup/<PROXMOX-NAME>-nvme0n1.img.zst && \
echo "==== ZSTD VERIFICATION PASSED ====="'
ssh user@<UBUNTU IP> \
'sha256sum /proxmox-backup/<PROXMOX-NAME>-nvme0n1.img.zst | tee /proxmox-
backup/<PROXMOX-NAME>-nvme0n1.img.zst.sha256'
```

Sanitizing the Disk

The Block Erase Function

Since the backup appeared to be successful, and the host was being sent out of possession, the next task was to securely sanitize the hard drive to prevent the data from being exposed. Using the information obtained from the serial number, the sanicap value, and the manufacturer's data sheet, the supported sanitization method was Block Erase. To summarize, this method erases the drive's NAND flash storage, which involves physically removing the electrical charge that stores the state. Once the charge state of the cell is erased, there is no longer a representation of the data stored for the controller to read. In this case, a Block Erase might be excessive, but methods such as formatting the drive are not sufficient for higher security environments.

Interacting with the Hardware

The nvme package interacts with the physical drive directly and instructed the controller to execute a Block Erase. This was represented by the sanact=0x02 flag. Checking the progress of the erase was also possible with nvme.

```
sudo nvme sanitize /dev/nvme0n1 --sanact=0x02
sudo nvme sanitize-log -H /dev/nvme0n1
```

The entire Block Erase operation only took a few seconds, and a successful log was provided. The disk was finally checked for remaining data, errors, and partitions.

```
sudo fdisk -l /dev/nvme0n1
sudo nvme smart-log /dev/nvme0n1 | grep -E
'critical_warning|media_errors|num_err_log_entries|percentage_used'
```

```

[ubuntu@ubuntu:~]$ sudo nvme sanitize-log -H /dev/nvme0n1
Sanitize Progress      (SPROG) : 65535
Sanitize Status       (SSTAT) : 0x101
[2:0] : Sanitize Operation Status : 0x1    Most Recent Sanitize Command Completed Successfully.
[7:3] : Overwrite Passes Completed : 0
[8:8] : Global Data Erased : 0x1    No user data has been written in the NVM subsystem and no PMR has been enabled in the NVM subsystem
[9:9] : Media Verification Canceled: 0    Not canceled

Sanitize Command Dword 10 Information (SCDW10) : 0x2
Estimated Time For Overwrite : 4294967295 (No time period reported)
Estimated Time For Block Erase : 4294967295 (No time period reported)
Estimated Time For Crypto Erase : 4294967295 (No time period reported)
Estimated Time For Overwrite (No-Deallocate) : 0
Estimated Time For Block Erase (No-Deallocate) : 0
Estimated Time For Crypto Erase (No-Deallocate) : 0
Estimated Time For Post-Verification Deallocation: 0
Sanitize State Information (SSI) : 0
[3:0] : Sanitize State : 0    Idle state

[ubuntu@ubuntu:~]$ sudo fdisk -l /dev/nvme0n1
Disk /dev/nvme0n1: 953.87 GiB, 1024209543168 bytes, 2000409264 sectors
Disk model: BG6 KIOXIA 1024GB
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk Identifier:

Device      Start      End      Sectors  Size Type
/dev/nvme0n1p1    34        2047        2014 1007K BIOS boot
/dev/nvme0n1p2   2048    2099199    2097152    1G EFI System
/dev/nvme0n1p3 2099200 1998585856 1996486657 952G Linux LVM
[ubuntu@ubuntu:~]$ sudo nvme smart-log /dev/nvme0n1 | grep -E 'critical_warning|media_errors|num_err_log_entries|percentage_used'
critical_warning      : 0
percentage_used       : 17%
media_errors          : 0
num_err_log_entries   : 0

```

As seen in the screenshot, the “Most recent sanitize command completed successfully”, the sanitize state was idle, and there haven’t been any writes in the NVM subsystem. The drive also reported no critical warnings, media errors, or error log entries. This disk had been successfully wiped, with no errors.